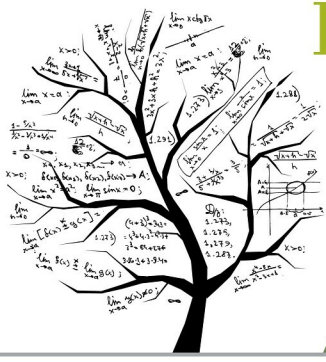
Exploring a Few Type Classes in Haskell



Haskell, an open source programming language, is the outcome of 20 years of research. It is named after the logician, Haskell Curry. It has all the advantages of functional programming and an intuitive syntax based on mathematical notation. Let's continue our exploration of Haskell.

In this article, we shall explore more type classes in Haskell. Consider the *Functor* type class:

```
class Functor f where
      fmap :: (a -> b) -> f a -> f b
```

It defines a function *fmap*, which accepts a function as an argument that takes input of *Type a* and returns *Type b*, and applies the function on every *Type a* to produce *Type b*. The *f* is a type constructor. An array is an instance of the *Functor* class and is defined as shown below:

```
instance Functor [] where
      fmap = map
```

The *Functor* type class is used for types that can be mapped over. Examples of using the *Functor* type class for arrays are shown below:

```
ghci> fmap length ["abc", "defg"]
[3,4]

ghci> :t length
length :: [a] -> Int

ghci> map length ["abc", "defg"]
[3,4]

ghci> :t map
map :: (a -> b) -> [a] -> [b]
```

An instance of a *Functor* class must satisfy two laws. First, it must satisfy the identity property where running the map over an ID must return the id. The term 'id' represents identity.

```
fmap id = id
```

For example:

```
ghci> id ["abc"]
["abc"]

ghci> fmap id ["abc"]
["abc"]
```

Second, if we compose two functions and *fmap* over it,